

Secure Password Manager

Computer Engineering Lab Final Project

By: Amr Shaarawy

- Stores credentials locally in encrypted form
- Uses file handling for persistence
- Generates secure passwords
- Implements master-password authentication
- Supports CSV import/export

Main Data Flow and System Architecture

- The program is layered into a disk storage layer that is encrypted, containing the master password and the saved password database, and a RAM layer that is in charge of the current run
- Authentication: the user is first asked the master password, if incorrect the access is denied, if correct he is shown a menu of operations
- Operations include generating/adding a password, exporting/importing a csv file from/to RAM (plain text), display on terminal, changing the master or resetting the database
- Addition of an entry or changing the master saves the changes to the disk layer

Data Structure in RAM

- Password entries are managed by means of a linked list that is constructed from the encrypted file at every run
- The struct PasswordNode contains 3 fields for password, user and domain, and a pointer for the next node
- The data here is in plain text for readability of display or exporting operations
- A new entry is added to the linked list and then saved encrypted in the passwords.dat file

Files Stored on Disk

Master password:

- The first use of the code uses a default password since there is no file to load [load_master()]
- If the master password is changed, the new password is saved encrypted in the file master.bin [save_master()]
- For the next use the saved master password is loaded from the file master.bin and decrypted for authentication

Password database:

- The function save_db() encrypts all the fields of the PasswordNode struct for each node of the list and writes them to the file passwords.dat for storage on the disk
- After writing they are decrypted back so they would stay in clear text on the RAM for display/export
- load_db() reads the passwords.dat file and decrypts a number (count header), then 3 fields of fixed size for each node up to the count limit

Encryption Logic

- Simple encryption is used as alternative to AES, the encryption/decryption operations are still symmetric
- Encrypts/decrypts by performing XOR on each character with a key character
- Applied before writing to the secure files on the disk
- Applied for decryption when loading the file

Functions met

- Manages authentication via a master password
- Adds entries via manual add or csv imports
- Displays entries on terminal or csv export
- Generates secure passwords and saves them
- Stores encrypted database of entries and encrypted master password
- Analyzes password strength

Thank you!